	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Carlos Chaves Mercado

Asignatura: Fundamentos de programación

Grupo: 8

No de Práctica(s): Práctica de estudio 07: Estructuras de selección

Integrante(s): Nolasco Ramírez Sofía

Díaz Rodríguez José Pablo

Cruz Gómez Lizette Jacqueline

Macay Martínez David

No. de lista o brigada: Mesa 6

Semestre: 2026-2

Fecha de entrega: 09 de marzo de 2026

Observaciones:

CALIFICACIÓN: _____

Objetivo :

El alumno elaborará programas en lenguaje C que incluyan las estructuras de selección if, if-else, switch y ternaria (o condicional) para la resolución de problemas básicos.

Introducción

La capacidad de un sistema para tomar decisiones de manera autónoma es fundamental en la ingeniería. Dentro del ámbito de la programación se implementan estructuras de control condicionales que permiten desviar el flujo de ejecución de un programa basándose en la evaluación diferentes expresiones lógicas o booleanas establecidas por el programador.
Las estructuras de control condicional permiten que un código responda de manera distinta ante diferentes entradas o estados del sistema. Existen 4 mecanismos esenciales que se utilizaron en la práctica y serán mencionados a continuación:

- Sentencia *if*: La forma más básica de control, que ejecuta un bloque de código solo si se cumple una condición específica.
- Sentencia *if - else*: Permite dividir el flujo, ofreciendo una ruta alternativa cuando la condición inicial resulta falsa.
- Estructura *switch*: Optimiza la toma de decisiones cuando existen múltiples caminos posibles basados en el valor de una sola variable, evitando la anidación excesiva de condicionales.
- Operador ternario "*? :*" : Es una sintaxis simplificada y compacta para realizar asignaciones condicionales rápidas, mejorando la legibilidad en expresiones sencillas.

Desarrollo

La práctica 7 se desarrolló mediante la elaboración de un programa en lenguaje C en el cual se integraron conceptos trabajados previamente en prácticas anteriores y estructuras de control condicional. A continuación, se presentarán ejemplos de distintos programas realizados usando diferentes estructuras de control, para distintos problemas.

Programa1.c

Este programa valida si el valor de la variable a es mayor al valor de la variable b.

Figura 1. Código fuente del programa 1 en lenguaje C

Programa2.c

Este programa comprueba que las condiciones son numéricas 0 → falso, ≠ 0 → verdadero.

```
/*****************************************************************

                    Online C Compiler.
            Code, Compile, Run and Debug C program online.
Write your code in this editor and press "Run" button to compile and exec

*****************************************************************

#include<stdio.h>
int main()
{
    if (0)
    {
        printf("Esta instrucción nunca se ejecuta\n");
        printf("porque la condición siempre es falsa (0).\n");

    }
    if (-38) // El bloque de código de esta estructura if
            // solo consta de una linea porque los comentarios // no son
            // La condición siempre es verdadera (diferente de 0)
            printf("Esta instrucción siempre se ejecuta.\n");
        return 0;
}
```

```
Esta instrucción siempre se ejecuta.

...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 2. Código fuente del programa 2 en lenguaje C

Programa3.c

Este programa permite validar si un número es par o impar. El número se lee desde la entrada estándar (el teclado).

```
/*****************************************************************

                    Online C Compiler.
            Code, Compile, Run and Debug C program online.
Write your code in this editor and press "Run" button to compile and exec

*****************************************************************

#include <stdio.h>
int main()
{
    int num;
    printf("Ingrese un número:\n");
    scanf("%d",&num);
    if ( num%2 == 0 )
        printf("El número %d es par.\n",num);
    else
        printf("El número %d es impar.\n",num);
    return 0;
}
```

```
Ingrese un número:
8
El número 8 es par.

...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 3. Código fuente del programa 3 en lenguaje C

Programa4.c

Este programa ordena en forma descendente tres valores enteros dados. Los valores se leen desde la entrada estándar (el teclado).

```c
/*****************************************************************************

                    Online C Compiler.
            Code, Compile, Run and Debug C program online.
Write your code in this editor and press "Run" button to compile and execute it.

*****************************************************************************/
#include <stdio.h>
int main() {
int uno, dos, tres;
printf ("Ingrese 3 números separados por espacios:\n"); scanf ("%d %d %d", &uno, &dos, &tres);
if (uno > dos)
{
if (dos > tres) {
printf("%d es mayor a %d que es mayor a %d\n", uno, dos, tres); }
else
{
if (uno > tres)
{
printf("%d es mayor a %d que es mayor a %d\n", uno, tres, dos);
}
else
{
printf("%d es mayor a %d que es mayor a %d\n", tres, uno, dos);
} }
}
else
{
if (dos > tres) {
if (tres > uno) {
printf("%d es mayor a %d que es mayor a %d\n", dos, tres, uno); }
else
{
```

```
Ingrese 3 números separados por espacios:
5 10 2
10 es mayor a 5 que es mayor a 2

...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 4. Código fuente del programa 4 en lenguaje C

Programa5.c

Este programa permite elegir una opción del menú a partir del carácter ingresado. La opción se lee desde la entrada estándar (el teclado).

```c
/*****************************************************************************

                    Online C Compiler.
            Code, Compile, Run and Debug C program online.
Write your code in this editor and press "Run" button to compile and execute it.

*****************************************************************************/
#include <stdio.h>
int main()
{
char op = '\0';
printf("\tMenú\n\n");
printf("Elegir la opción deseada\n"); printf("a) Ingresar\n");
printf("b) Registrarse\n"); printf("c) Salir\n"); scanf("%c",&op);
switch(op)
{
default:
printf("Opción no válida.\n"); break;
case 'a':
printf("Se seleccionó 'Ingresar'.\n"); break;
case 'b':
printf("Se seleccionó 'Registrarse'.\n"); break;
case 'c':
printf("Se seleccionó 'Salir'.\n"); break;
}
return 0;
}
```

```
        Menú

legir la opción deseada
a) Ingresar
b) Registrarse
c) Salir
c
e seleccionó 'Ingresar'.

...Program finished with exit code 0
ress ENTER to exit console.
```

Figura 5. Código fuente del programa 5 en lenguaje C

Programa6.c

Este programa permite elegir una opción del menú a partir del entero.

```c
/*****************************************************************************

                    Online C Compiler.
            Code, Compile, Run and Debug C program online.
Write your code in this editor and press "Run" button to compile and execute it.

*****************************************************************************/
#include <stdio.h>
int main() {
int op;
printf("\tMenú\n\n");
printf("Elegir la opción deseada\n"); printf("1) Ingresar\n");
printf("2) Registrarse\n"); printf("3) Salir\n"); scanf("%d",&op);
switch(op) {
case 1:
printf("Se seleccionó 'Ingresar'\n"); break;
case 2:
printf("Se seleccionó 'Registrarse'\n"); break;
case 3:
printf("Se seleccionó 'Salir'\n"); break;
default:
printf("Opción no válida\n");
}
return 0; }
```

```
        Menú

Elegir la opción deseada
1) Ingresar
2) Registrarse
3) Salir
3
Se seleccionó 'Salir'


...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 6. Código fuente del programa 6 en lenguaje C

Programa7.c

Este programa crea diversas variables tipo enum (enumerador) y permite visualizar la manera en la que se maneja el tipo de dato.

```c
/*****************************************************************************

                    Online C Compiler.
            Code, Compile, Run and Debug C program online.
Write your code in this editor and press "Run" button to compile and execute it.

*****************************************************************************/
#include <stdio.h>
int main()
{
    // declaración de la enumeración
enum boolean {NO, YES};
    // declaración de una variable tipo enumeración
enum boolean valorBooleano;
valorBooleano = YES;
// Se comprueba que el valor de una enumeración es entero
printf("%d\n", valorBooleano);
// Se comprueba que el valor de una enumeración se puede reasignar
enum diasSemana {LUNES, MARTES, MIERCOLES=5, JUEVES, VIERNES};
printf("\n%d", LUNES);
printf("\n%i", MARTES);
printf("\n%d", MIERCOLES);
printf("\n%i", JUEVES);
printf("\n%d\n", VIERNES);
return 0;
}
```

```
1

0
1
5
6
7


...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 7. Código fuente del programa 7 en lenguaje C

Programa8.c

Este programa permite elegir una opción del menú a partir del entero ingresado. La opción se lee desde la entrada estándar (el teclado).

```c
#include <stdio.h>
int main()
{
    // Los valores de una enumeración son enteros y constantes
    enum diasSemana {LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, D
    printf("Ingrese el día de la semana.\n");
    printf("1) Lunes\n");
    printf("2) Martes\n");
    printf("3) Miércoles\n");
    printf("4) Jueves\n");
    printf("5) Viernes\n");
    printf("6) Sábado\n");
    printf("7) Domingo\n");
    scanf("%d", &op);
    switch(op-1)
{
    case LUNES:
    case MARTES:
        printf("Inicio de semana.\n");
        break;
    case MIERCOLES:
        printf("Mitad de semana.\n");
        break;
    case JUEVES:
```

```
Ingrese el día de la semana.
1) Lunes
2) Martes
3) Miércoles
4) Jueves
5) Viernes
6) Sábado
7) Domingo
4
¡Casi inicia el fin de semana!

...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 8. Primer parte del código fuente del programa 8 en lenguaje C

```c
    scanf("%d", &op);
    switch(op-1)
{
    case LUNES:
    case MARTES:
        printf("Inicio de semana.\n");
        break;
    case MIERCOLES:
        printf("Mitad de semana.\n");
        break;
    case JUEVES:
        printf("¡Casi inicia el fin de semana!\n");
        break;
    case VIERNES:
    case SABADO:
        printf("¡Fin de semana!\n");
        break;
    case DOMINGO:
        printf("Día de descanso.\n");
        break;
    // No se necesita default

    }
    return 0x0; // Valor entero en hexadecimal
}
```

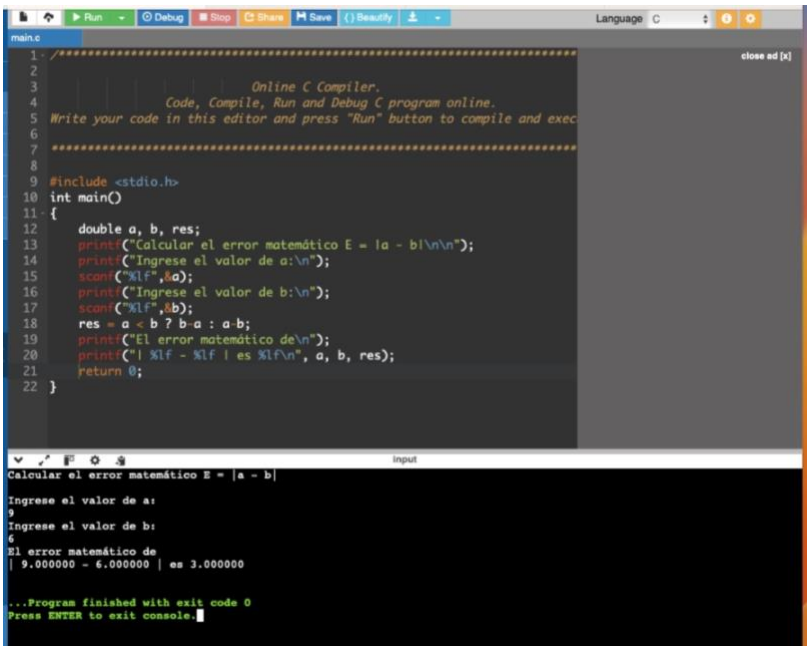
```
Ingrese el día de la semana.
1) Lunes
2) Martes
3) Miércoles
4) Jueves
5) Viernes
6) Sábado
7) Domingo
4
¡Casi inicia el fin de semana!

...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 8.1 Primer parte del código fuente del programa 8 en lenguaje C

Programa9.c

Este programa permite calcular el error matemático a partir de dos valores (a y b) ingresados desde la entrada estándar (el teclado), a partir de la fórmula: E = |a – b|. Donde a es el valor real y b es el valor aproximado o viceversa.



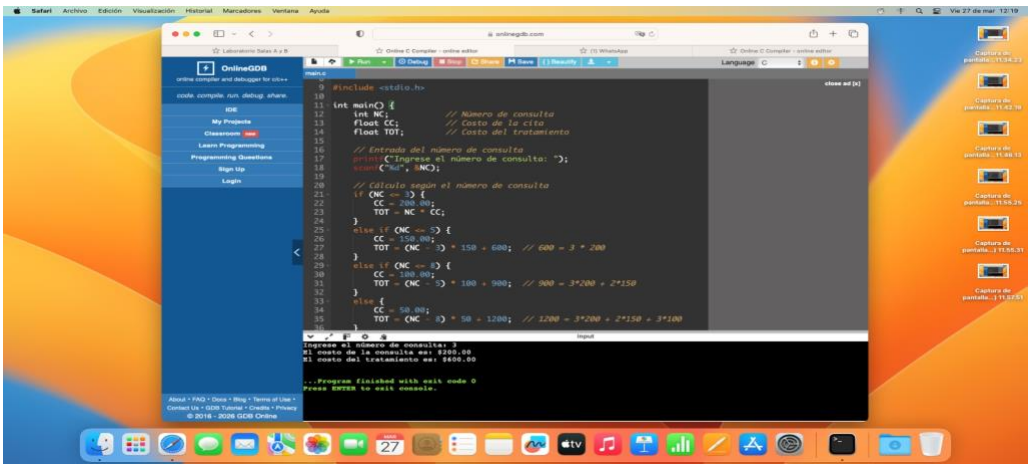
Ejemplo 1:

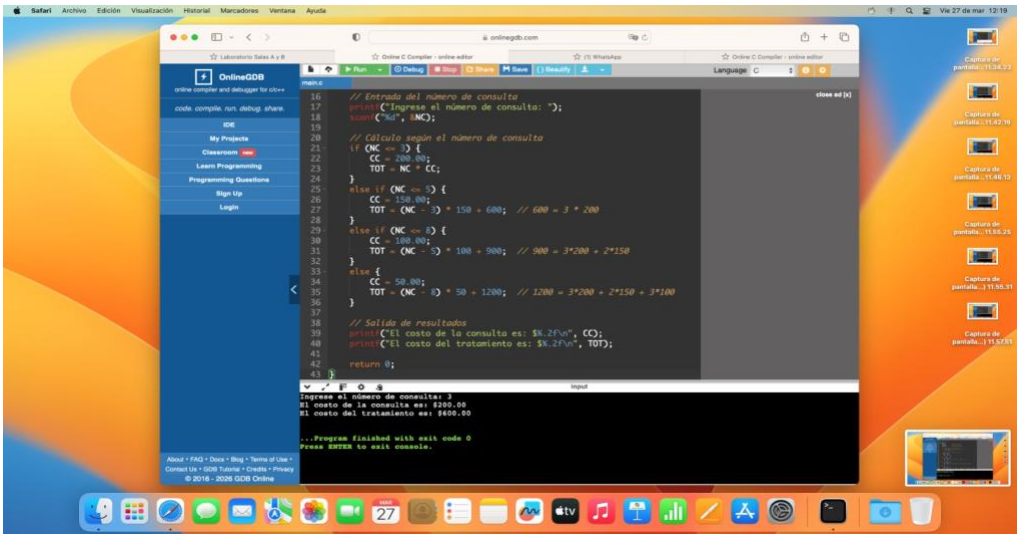
El consultorio del Dr. Lorenzo T. Mata Lozano tiene como política cobrar la consulta con base en el número de cita, de la siguiente forma:

- *Las tres primeras citas a $200.00 c/u.*
- *Las siguientes dos citas a $150.00 c/u.*
- *Las tres siguientes citas a $100.00 c/u.*
- *Las restantes a $50.00 c/u, mientras dure el tratamiento.*

Se requiere un algoritmo para determinar:

a) *Cuánto pagará el paciente por la cita.*
b) *El monto de lo que ha pagado el paciente por el tratamiento.*



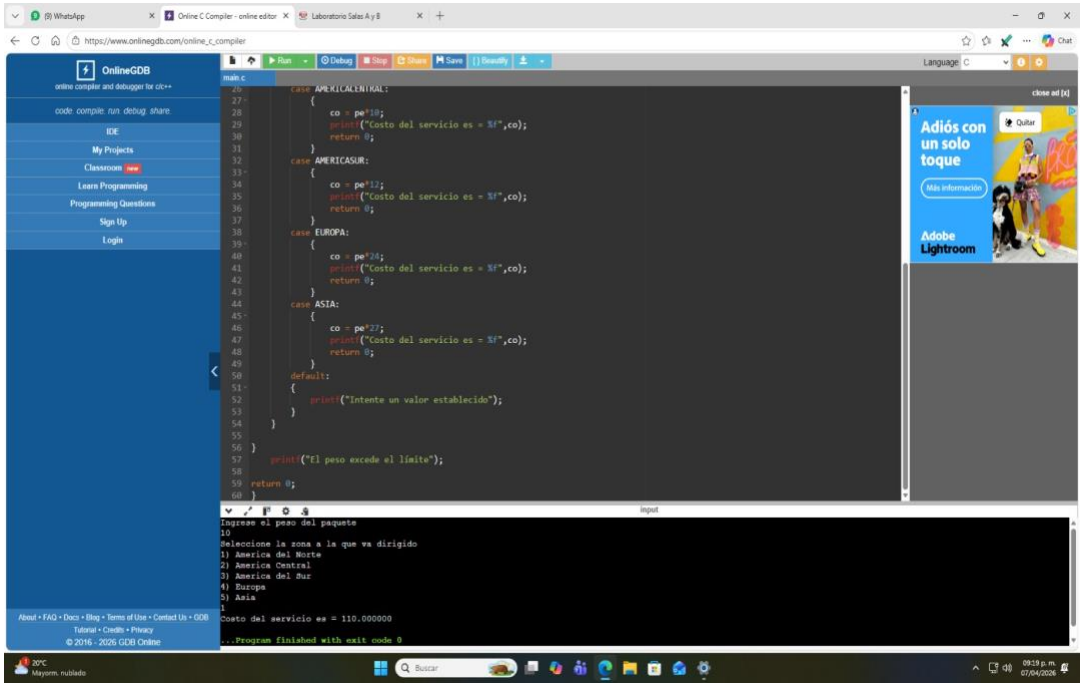


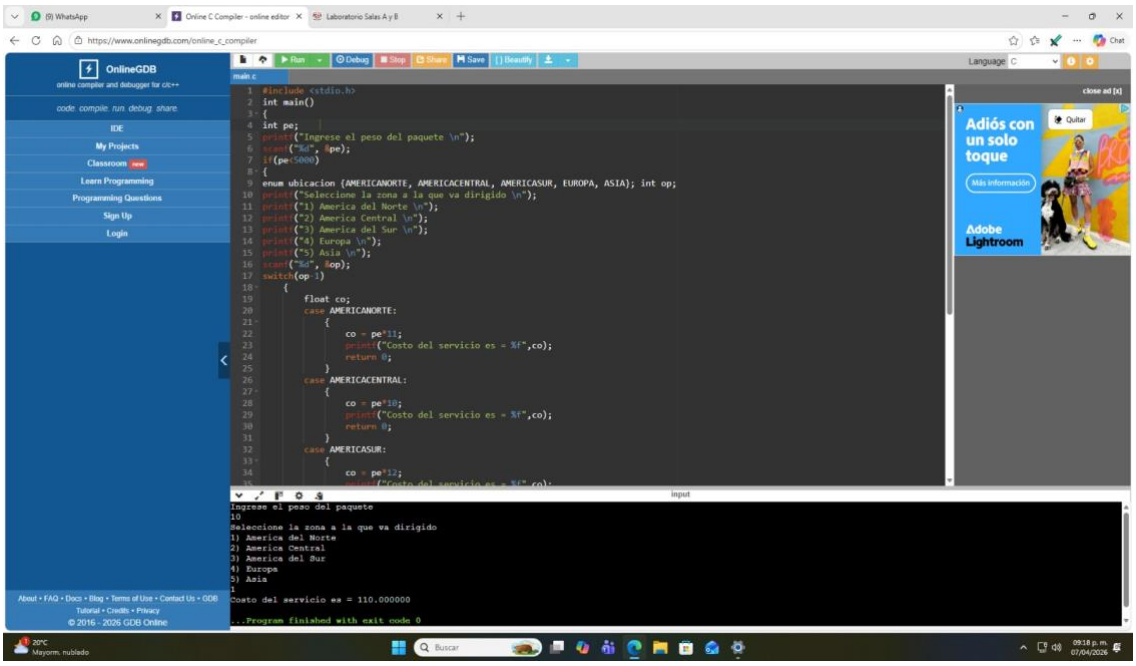
Ejemplo 2:

Una compañía de paquetería internacional tiene servicio en algunos países de América del Norte, América Central, América del Sur, Europa y Asia. El costo por el servicio de paquetería se basa en el peso del paquete y la zona a la que va dirigido. Lo anterior se muestra en la siguiente tabla:

Zona	*Ubicación*	*Costo/gramo*
1.	*América del Norte*	*$11.00*
2.	*América Central*	*$10.00*
3.	*América del Sur*	*$12.00*
4.	*Europa*	*$24.00*
5.	*Asia*	*$27.00*

Parte de su política implica que los paquetes con un peso superior a 5 kg no son transportados, esto por cuestiones de logística y de seguridad. Realice un algoritmo para determinar el cobro por la entrega de un paquete o, en su caso, el rechazo de la entrega; represéntelo mediante diagrama de flujo, y pseudocódigo.





Análisis de resultados:

Programa1.c

El uso de una estructura lógica donde una variable a es mayor a una variable b constituye el núcleo de la toma de decisiones algorítmica en entornos profesionales, permitiendo que un sistema actúe de manera autónoma ante variaciones de datos. En ámbitos como la ingeniería industrial y l ciberseguridad, esta estructura es fundamental para el control de procesos y la gestión de riesgos, ya que permite monitorear umbrales críticos y activar protocolos de seguridad instantáneos cuando los valores operativos superan los límites permitidos. Asimismo, su aplicación en el sector financiero y el Business Intelligence garantiza la integridad operativa al automatizar la detección de anomalías, como excedentes presupuestarios o transacciones que sobrepasan fondos disponibles, transformando una simple comparación matemática en una herramienta estratégica para la optimación de recursos y la respuesta inmediata en tiempo real

Programa2.c

Este programa ejemplifica el concepto fundamental de evaluación de verdad lógica en programación, donde cualquier valor numérico distinto de cero se interpreta como verdadero y únicamente el cero se considera falso. Es la base de la gestión de estados y banderas (flags) en sistemas operativos y desarrollo de controladores (drivers), permitiendo que el software verifique rápidamente si un recurso está disponible o si un proceso ha retornado un código de error sin necesidad de comparaciones complejas.

Programa3.c

Se utiliza el operador residuo o módulo (%) para implementar una lógica de clasificación binaria, técnica esencial en la computación para la organización y filtrado de datos. En un entorno profesional, este análisis de paridad se aplica en la criptografía y seguridad informática para algoritmos de cifrado, así como en la optimización de hardware, donde permite distribuir cargas de trabajo de manera equitativa entre procesadores mediante la asignación de tareas a hilos pares o impares. Asimismo, es una herramienta fundamental en el procesamiento de señales y gráficos por computadora, donde se utiliza para aplicar efectos de tramado o para alternar estilos en interfaces de usuario masivas, demostrando cómo una operación aritmética simple permite estructurar sistemas complejos mediante la segmentación lógica de la información.

Programa4.c

Este código implementa un algoritmo de ordenamiento por comparación directa, el cual representa la base de la jerarquización de datos y la priorización en sistemas computacionales. En el ámbito profesional, esta estructura de condiciones anidados es fundamental para el desarrollo de motores de búsqueda, sistemas de gestión de bases de datos y herramientas de análisis financiero, donde se requiere organizar volúmenes masivos de información bajo criterios de relevancia o magnitud de forma descendente. Además, este tipo de lógica es el pilar de la inteligencia de negocios (Business Intelligence) para la creación de rankings y la clasificación de indicadores de desempeño (KPIs), permitiéndoosla que el software identifique automáticamente jerarquías críticas y facilite la toma de decisiones estratégicas al presentar la información de mayor impacto de manera prioritaria.

Programa5.c

Implementa una estructura de selección múltiple mediante la sentencia switch, la cual representa la base del diseño de interfaces de usuario y sistemas de navegación interactivos. Esta lógica es fundamental para el desarrollo de sistemas operativos y menús de configuración, donde permite gestionar de manera eficiente múltiples rutas de ejecución a partir de una única entrada del usuario, optimizando el rendimiento de procesador en comparación con múltiples condiciónales anidado. Asimismo, esta estructura es esencial en la creación de máquinas de estado en sistemas embebidos y controladores industriales, permitiendo que un dispositivo cambie su modo de operación (como encendido, espera o diagnóstico) de forma segura y estructurada.

Programa6.c

Utiliza la instrucción switch con selectores numéricos de tipo entero, una estructura fundamental para la implementación de menús de navegación y sistemas de control jerárquico. Profesionalmente, esta técnica se emplea en el desarrollo de interfaces de línea de comandos (CLI) y sistemas POS (puntos de venta), donde la entrada de un código número permite ejecutar funciones específicas de forma rápida y eficiente, minimizando la carga computacional en comparación con el procesamiento de cadenas de texto.

Programa7.c

Implementa el uso de tipo de datos enumerados (enumeró), una herramienta fundamental para mejorar la legibilidad y el mantenimiento del código mediante la abstracción de valores numéricos en etiquetas con significado semántico. Profesionalmente, esta técnica se utiliza en el desarrollo de sistemas de software robustos y grandes infraestructuras de código, donde permite definir estados de programa, categorías o configuraciones especificas (como días de la semana o estados de una conexionarme) de forma clara y menos propensa a errores que el uso de constantes mágicas.

Programa8.c

Integración avanzada de tipos enumerados (enum) con estructuras de control switch, una combinación esencial para el desarrollo de lógica de negocio basada en estados o categorías. En el ámbito profesional, esta metodología se utiliza en la creación de sistemas de gestión de flujos de trabajo (Workflow Management) y planificadores de tareas, donde el uso de etiquetas descriptivas en lugar de números planos reduce significativamente los errores humanos durante la codificación y facilita el mantenimiento a largo plazo. Al mapear entradas del usuario directamente a constantes enumeradas, el software garantiza una validación de dominio estricta, asegurando que solo se procesen comandos válidos y permitiendo una expansión modular del sistema. Esta arquitectura es un estándar en el desarrollo de motores de lógica y aplicaciones administrativas, ya que permite traducir procesos del mundo real —como la gestión de un calendario o el estado de un pedido— en un código estructurado, altamente legible y profesionalmente escalable.

Programa9.c

Este programa implementa el cálculo del error absoluto mediante el uso del operador ternario, una estructura compacta y eficiente para la toma de decisiones rápidas en programación. En el ámbito

profesional, este tipo de algoritmos es fundamental en la metrología, la ingeniería de control y el análisis numérico, donde se requiere medir con precisión la desviación entre un valor teórico y uno experimental para validar la calidad de procesos industriales. Al utilizar la lógica de valor absoluto ($E=|a-b|$), el software garantiza que la magnitud del error sea siempre positiva, independientemente del orden de los datos, lo cual es un estándar crítico en el control de calidad y la estadística aplicada para determinar tolerancias y márgenes de precisión. Esta aproximación técnica demuestra cómo una sola línea de código puede resolver problemas de normalización de datos, facilitando el desarrollo de sistemas de monitoreo robustos y la toma de decisiones basadas en la cuantificación de la incertidumbre.

Ejemplo 1:

En este ejercicio, el programa gestionó el cálculo de pagos mediante una estructura de selección múltiple basada en el número de cita ingresado. El sistema utilizó una variable entera para recibir el identificador de la cita y, a través de sentencias condicionales anidadas o una estructura de control de flujo, asignó un costo unitario diferenciado según los rangos establecidos: $200.00 para las primeras tres consultas, $150.00 para las dos siguientes, $100.00 para el tercer bloque y $50.00 para cualquier cita posterior.

Durante la ejecución, el algoritmo no solo determinó el monto individual de la consulta actual, sino que también realizó una operación de acumulación para calcular el gasto total del tratamiento. Se observó que el programa aplicó fórmulas de suma aritmética para integrar los costos de los rangos anteriores al valor de la cita presente, garantizando así que el dato de salida reflejara el histórico de pagos del paciente. Este proceso permitió verificar la eficiencia de las estructuras de selección para segmentar datos numéricos y ejecutar cálculos financieros precisos basados en condiciones variables de entrada.

Ejemplo 2:

En este apartado se evaluó el funcionamiento de un algoritmo diseñado para la gestión de envíos internacionales basándose en dos criterios críticos: la zona de destino y la masa del paquete. El programa inició el proceso mediante una estructura de validación que comparó el peso ingresado (en gramos) contra el límite de seguridad establecido de 5000 gramos. Durante las pruebas, se comprobó que el sistema actúa como un filtro de seguridad, rechazando automáticamente cualquier paquete que exceda dicho umbral y emitiendo un mensaje de restricción logística, lo que evita que el programa realice cálculos innecesarios para envíos no permitidos.

Para los casos donde el peso resultó válido, el sistema ejecutó una estructura de selección múltiple (como `switch` o `if-else` anidados) para identificar la zona geográfica seleccionada. El algoritmo asoció cada ubicación con su respectiva tarifa constante por gramo ($11.00, $10.00, $12.00, $24.00 o $27.00), realizando posteriormente una operación aritmética de multiplicación para determinar el costo total del servicio. Los resultados demostraron que el software es capaz de integrar validaciones de seguridad con lógica de asignación de costos, garantizando que el cobro final sea proporcional tanto a la región de destino como a la magnitud física del objeto transportado.

Conclusión

Diaz Rodríguez José Pablo:

Nolasco Ramírez Sofia: El desarrollo de esta práctica permitió comprobar la importancia de las estructuras de selección como el eje fundamental para la toma de decisiones dentro de un algoritmo. A través de la implementación de sentencias if, if-else y switch en lenguaje C, se verificó que un programa puede dejar de ser una secuencia lineal de instrucciones para convertirse en un sistema dinámico capaz de evaluar condiciones lógicas y responder de manera diferenciada ante diversos tipos de datos de entrada.

Durante las actividades, se constató que la correcta estructuración de estas sentencias no solo garantiza que el software cumpla con su función técnica (como calcular costos de consultas médicas o validar el peso de paquetes internacionales), sino que también establece mecanismos de seguridad y control de errores al restringir el flujo de ejecución solo a escenarios válidos. La experiencia en el laboratorio demostró que el manejo preciso de los operadores de relación y la comprensión de los valores de verdad son habilidades críticas en ingeniería, pues permiten construir soluciones robustas que modelan comportamientos complejos y resuelven problemas de la vida cotidiana de forma automatizada y eficiente.

Cruz Gómez Lizette Jacqueline: Las estructuras de selección son, en esencia, las que permiten que un programa deje de ser una lista rígida de pasos y se convierta en una herramienta capaz de "pensar" y decidir. Su importancia en el desarrollo de software profesional es absoluta, ya que son el recurso que permite gestionar la incertidumbre y las variables del mundo real, adaptando la respuesta del sistema según los datos que recibe en cada momento. Ya sea mediante un if para validar una condición crítica de seguridad, un switch para organizar menús complejos, o el uso de enums para que el código sea más humano y menos propenso a errores, estas estructuras garantizan que el flujo de información sea coherente y eficiente. En conclusión, dominar estas herramientas es lo que permite construir sistemas robustos y escalables, capaces de resolver problemas complejos mediante una lógica organizada, limpia y, sobre todo, funcional para cualquier entorno operativo.

Macay Martínez David: A través del desarrollo de esta práctica, se logró realizar diferentes programas ejecutables en lenguaje C, haciendo uso de las herramientas de edición, compilación y ejecución. El uso de distintas estructuras de control condicionales permitió comprender la manera en que se ejecuta un proceso lógico de selección, además de la aplicación de un programa computacional para solucionar problemas cotidianos, a partir del uso de sentencias como: *if, if-else* y la estructura *switch*.

Bibliografía

El lenguaje de programación C. Brian W. Kernighan, Dennis M. Ritchie, segunda edición, USA, Pearson Educación 1991.